Introduction to Computer Systems

Lecture 13: Distributed Systems

Remote Procedure Calls, Distributed File Systems, and Failure Handling

Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Contents

1 Introduction **3**

2 Why Distributed Systems Exist **3**
2.1 More computational power . . . 3
2.2 Information sharing and collaboration . . . 4
2.3 Modularity and independent development . . . 4
2.4 Incremental scaling and administrative independence . . . 5

3 What Is a Distributed System? **5**
3.1 Shared-nothing architecture . . . 5
3.2 Single-system image . . . 6
3.3 Middleware . . . 6

4 Communication Building Block: Remote Procedure Calls **7**
4.1 From sockets to protocols . . . 7
4.2 The RPC abstraction . . . 8
4.3 RPC performance is not local-call performance . . . 9
4.4 RPC failure semantics . . . 9

5 Distributed-System Challenges **10**
5.1 Worse availability can result from more machines . . . 10
5.2 Reliability and failure-handling code . . . 11
5.3 Security . . . 11
5.4 The core consistency question . . . 11

6 Distributed File Systems **12**
6.1 The goal: transparent remote files . . . 12
6.2 A simple design: forward every operation . . . 13
6.3 Caching to reduce network load . . . 14
6.4 Block-level versus file-level network storage . . . 15

7 NFS: Simplicity, Statelessness, and Weak Consistency **16**
7.1 Write-through caching . . . 16
7.2 Weak cache consistency . . . 16
7.3 Multiple writers . . . 17
7.4 Pros and cons . . . 17

8 AFS: Whole-File Caching, Callbacks, and Session Semantics **17**
8.1 Design goals . . . 17
8.2 AFS assumptions . . . 18
8.3 Whole-file caching . . . 18
8.4 Callbacks . . . 19

9 File Access Consistency **19**
9.1 One-copy semantics in a local Unix file system . . . 19
9.2 NFS semantics . . . 20
9.3 AFS session semantics . . . 20
9.4 Why not use ordinary file sharing for collaborative editing? . . . 22

10 Failure Handling in Distributed File Systems **22**
10.1 Partial failure in a file system . . . 22
10.2 NFS stateless server design . . . 23
10.3 Why explicit offsets matter . . . 23
10.4 NFS mount behavior: wait or return error . . . 24
10.5 AFS failure handling and soft state . . . 24
10.6 Client crashes . . . 24

11 Disconnected Operation: Coda **25**
11.1 Hoarding . . . 25
11.2 Emulation . . . 25
11.3 Reintegration . . . 25

12 Consistency Models and Design Philosophy **26**
12.1 ACID versus BASE as design attitudes . . . 26
12.2 Where should conflicts be resolved? . . . 26
12.3 The design checklist . . . 27

13 Worked Examples **27**
13.1 Example 1: stale reads in NFS . . . 27
13.2 Example 2: last close wins in AFS . . . 27
13.3 Example 3: retry-safe positioned reads . . . 28
13.4 Example 4: duplicate suppression for a non-idempotent RPC . . . 28

14 Summary **28**

15 Quizzes & Exercises **30**
15.1 Short-Answer Quiz . . . 30
15.2 Applied Exercises . . . 30
15.3 Essay Questions . . . 31
15.4 Suggested Answers to the Short-Answer Quiz . . . 31

1 Introduction

Modern computer systems are rarely single isolated machines. A phone synchronizes a calendar with cloud services; a web application is assembled from many independently deployed services; a large machine-learning training job uses thousands of accelerators; and a car contains many small controllers communicating through an internal network. Even a single laptop has become a small parallel system because it contains multiple CPU cores, accelerators, storage controllers, wireless devices, and background services. Distributed systems are therefore not a niche topic: they are a normal way of building large, reliable, collaborative, and scalable software.

This lecture studies distributed systems through one of the most concrete abstractions in computer systems: files. A distributed file system tries to make a remote disk look like a local file system. That goal sounds simple, but it immediately exposes the hard questions of distributed systems. Which machine has the authoritative copy of a file? What should a client cache? When should cached data be invalidated? What happens if a server crashes after performing an operation but before sending the reply? Can a client retry the operation safely? Should a program block until a server comes back, or should the file system return an error?

The lecture has four major parts. First, we define what makes a system distributed and why one would build such a system. Second, we review remote procedure calls (RPCs), the communication building block that allows software modules on different machines to cooperate. Third, we use distributed file systems as an extended case study, focusing on the Network File System (NFS) and the Andrew File System (AFS). Finally, we examine failure handling, statelessness, idempotency, soft state, and disconnected operation in Coda.

Definition

A **distributed system** is a collection of independent computers that appears to its users as a single coherent system. The phrase "appears" is essential: the implementation contains many processors, many memories, many operating systems, many disks, and many failure modes, but the user-facing abstraction attempts to be one system.

Scope of This Lecture

The transaction material on database queries, write-ahead logging, serializability, conflict serializability, and two-phase locking belongs primarily to the database transaction lecture. Here we use those ideas only as background: distributed systems face a related consistency problem, but the main subject of this note is the content of the distributed-systems lecture - RPC, distributed file systems, NFS, AFS, and failure handling.

2 Why Distributed Systems Exist

2.1 More computational power

One traditional reason for distributed systems is performance. A single processor cannot increase clock frequency indefinitely, because power density and heat become limiting factors. Hardware designers responded by adding more cores and specialized accelerators, and system designers responded by distributing work across many machines. High-performance computing clusters, AI training clusters, search engines, storage systems, and analytics platforms all exploit this idea.

Performance aggregation, however, is not automatic. A workload must be decomposed into parallel tasks; intermediate data must be exchanged; stragglers must be handled; and failures must not destroy all progress. The larger the system, the more likely it is that at least one component is slow or broken at any moment.

Deep Dive

If a component has mean time between failures M and a job requires n such components to remain alive simultaneously, a simplified independent-failure model gives an aggregate failure rate of about n/M. The expected time until some component fails is therefore roughly M/n. This crude calculation explains why a GPU that individually appears reliable can become a frequent source of failure when a job depends on tens of thousands of GPUs at once.

2.2 Information sharing and collaboration

A more common everyday reason for distributed systems is information sharing. A user stores a document in a cloud drive not merely because the local disk is full, but because the document should be accessible from a laptop, phone, desktop, and perhaps by collaborators. Calendars, mailboxes, chat systems, code repositories, photo libraries, and collaborative documents are all distributed systems whose primary purpose is shared state.

Sharing creates immediate consistency questions. If Alice edits a file and Bob opens it one second later, should Bob see the update? If Bob already has the file open, should the contents change underneath his application? If Alice edits offline while Bob edits online, should the system merge the changes, choose one version, or ask the users? A distributed system cannot answer these questions solely with mechanisms; it must also define semantics.

2.3 Modularity and independent development

Distributed systems are also a software-engineering technique. A large e-commerce platform can be decomposed into services for accounts, inventory, cart management, recommendations, advertising, payment, fulfillment, fraud detection, logging, and analytics. Each service can be developed and tested by a smaller team, can use a database appropriate to its workload, and can expose an explicit interface to other services.

This modularity is especially valuable when components are supplied by different organizations. A car is a useful example. Even a conventional car may contain dozens of electronic control units for braking, steering, engine control, climate control, infotainment, sensors, and safety systems. Those controllers may be built by different suppliers and communicate through a vehicle network such as a CAN bus. The car manufacturer specifies messages and response semantics; each supplier controls the internal implementation of its component.

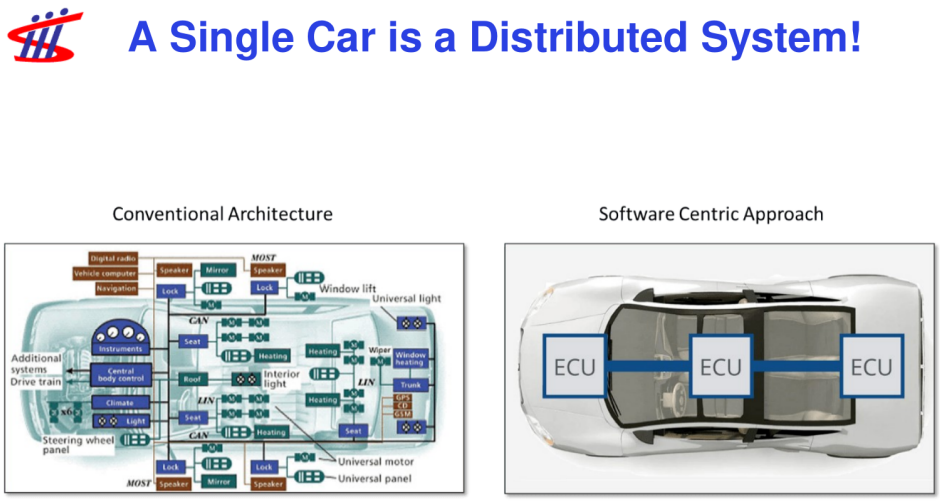


Figure 1: A car as a distributed system. A vehicle contains multiple independently designed controllers that coordinate through messages rather than through shared memory. The example illustrates that distribution is not only a data-center phenomenon; it is also a modularity strategy for embedded systems.

2.4 Incremental scaling and administrative independence

A distributed architecture lets a service add power incrementally. A new web service may begin with one server and a small database. If traffic increases, the operator can add web servers, storage servers, caches, load balancers, and replicas. Buying capacity gradually is economically better than purchasing a huge cluster before knowing whether the service will attract users.

Administrative independence is another motivation. Different users or organizations can control different machines and still participate in one system. The Internet is the largest example: it contains many administrative domains, but common protocols allow communication across them. Distributed file systems and cloud services similarly allow users to access resources that are administratively outside their local machine.

3 What Is a Distributed System?

A distributed system differs from a mere network. A network focuses on connectivity among endpoints. A distributed system uses that connectivity to provide a coherent service. The implementation may contain many independent computers, but the goal is to present a higher-level abstraction that users and applications can understand.

3.1 Shared-nothing architecture

Many distributed systems use a **shared-nothing architecture**. Each node owns its own CPU, memory, storage devices, operating system, and local failure domain. Nodes do not read or write one another's memory directly. They communicate by sending messages over a network or another communication medium.

This design scales because one can add more nodes. It also creates fundamental difficulty: there is no single memory location containing the global truth. Every fact about the system - which server stores a file, which cache is fresh, whether a request has executed, whether a node is down or merely slow - must be inferred from messages that may be delayed, lost, duplicated, or reordered.

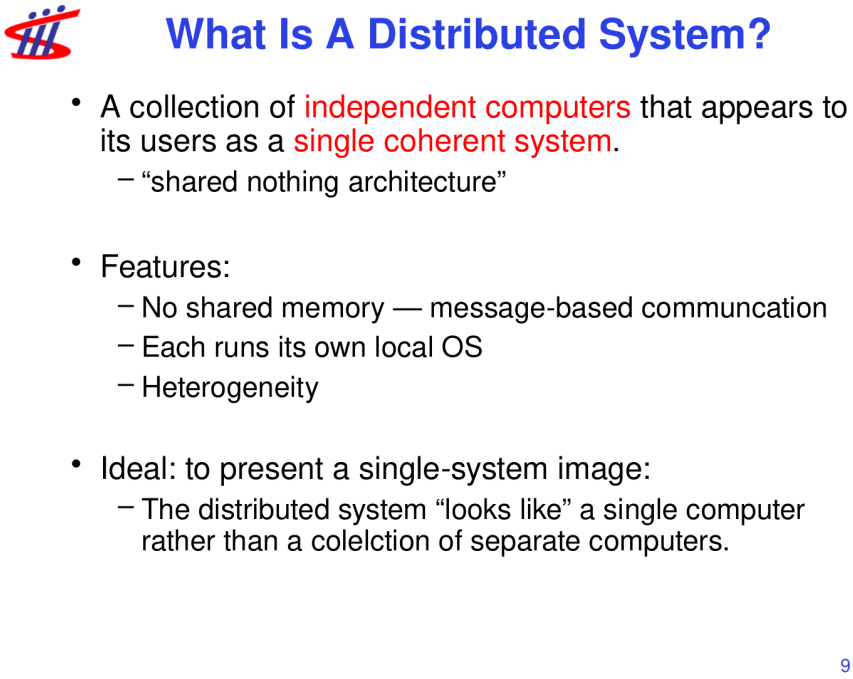


Figure 2: Definition of a distributed system. The slide emphasizes independent computers, lack of shared memory, local operating systems, heterogeneity, and the single-system image.

3.2 Single-system image

A **single-system image** is the illusion that many machines behave like one machine. A search engine provides one search box; the user need not know how many index servers, ranking services, storage systems, caches, and data centers participate. A cloud storage service presents one directory tree; the user need not know which blocks are stored on which disks.

This illusion is valuable, but it is not always desirable. Some systems deliberately expose distribution. Git exposes replicas, branches, commits, and merges. Kubernetes exposes pods, nodes, services, and failures to operators. Distributed transparency is useful only when the hidden details do not change the decisions that users or applications should make.

3.3 Middleware

The layer that hides distribution is often called **middleware**. Middleware runs above each local operating system and provides a uniform interface across machines. It may implement naming, authentication, service discovery, RPC, caching, replication, distributed locking, transactions, monitoring, or failure recovery.

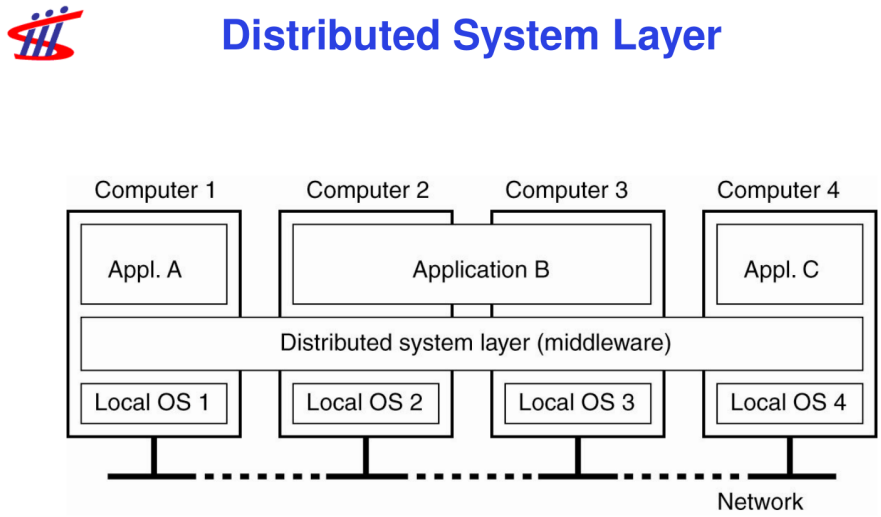


Figure 3: Middleware as the distributed-system layer. Each machine still has its local operating system; the middleware layer runs on the participating machines and presents a uniform interface to applications.

Middleware is helpful because applications should not repeatedly implement raw sockets, message framing, serialization, retry logic, load balancing, and authentication. Middleware is dangerous when it promises more transparency than the system can actually provide. A remote call cannot be exactly the same as a local function call: it is slower, can time out, can execute without returning, and can fail independently of the caller.

4 Communication Building Block: Remote Procedure Calls

4.1 From sockets to protocols

At the operating-system interface, network communication is exposed through sockets. A server creates a socket, binds it to an address and port, listens for connections, and accepts clients. A client creates a socket and connects to the server's address and port. After connection establishment, both sides exchange bytes.

A byte stream is too low-level for most application programming. It does not say which bytes represent an integer, which bytes represent a string length, where a record ends, or which byte order is used. A protocol must specify both **syntax** and **semantics**: the layout of messages and the meaning of each message.

Definition

A **protocol** is an agreement on how to communicate. Its syntax specifies message formats, field order, and encodings. Its semantics specifies what each message means and what actions should be taken when a message is sent, received, lost, retried, or times out.

4.2 The RPC abstraction

A remote procedure call makes communication look like a function call. The client invokes a local procedure-like interface; a generated client stub marshals the arguments into bytes; the bytes are sent over the network; a server stub unmarshals the arguments; the real server function executes; the result is marshaled back to the client.

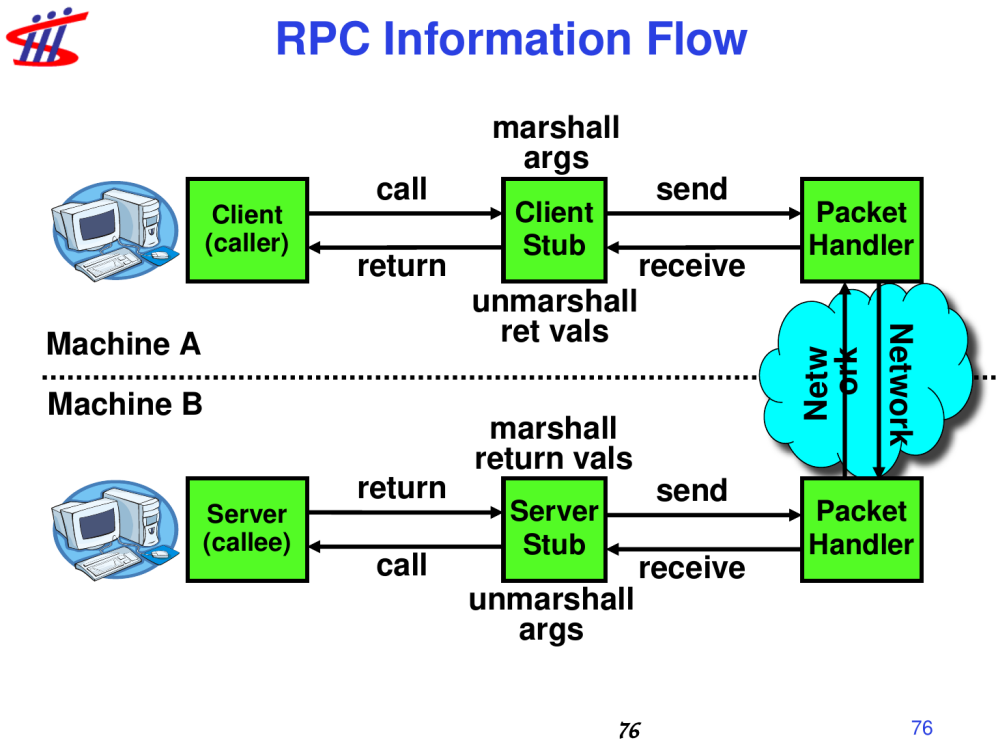


Figure 4: RPC information flow. The client calls a generated stub, which marshals arguments and sends a request. The server stub unmarshals the arguments, invokes the server procedure, marshals the return value, and sends a reply.

Definition

Marshalling or **serialization** converts typed values into a canonical byte representation suitable for transmission. **Unmarshalling** or **deserialization** reconstructs typed values from that byte representation.

A well-designed RPC system uses an interface definition language (IDL). The IDL declares services, methods, request messages, reply messages, field names, field types, and field numbers. A compiler then generates client and server stubs in the target language.

```
syntax = "proto3";

service FileService {
  rpc ReadAt(ReadAtRequest) returns (ReadAtReply);
  rpc WriteAt(WriteAtRequest) returns (WriteAtReply);
}

message ReadAtRequest {
  uint64 inode  = 1;
  uint64 offset = 2;
  uint64 length = 3;
}

message WriteAtRequest {
  uint64 inode  = 1;
  uint64 offset = 2;
  bytes data    = 3;
  uint64 request_id = 4;  // duplicate suppression
}
```

Listing 1: A minimal RPC-style service definition using Protocol Buffers syntax

The important point is not the particular syntax. The important point is that both endpoints are generated from the same interface definition. This avoids the fragile practice of manually casting byte arrays in C or C++ and hoping the sender and receiver interpret the layout in the same way.

4.3 RPC performance is not local-call performance

RPC is an interface abstraction, not a performance equivalence. A local function call may take nanoseconds. A same-machine RPC may take microseconds because it crosses process boundaries and performs serialization. A network RPC may take microseconds to milliseconds or longer, depending on network latency, queueing, packet loss, server load, and retry behavior. Programmers must therefore be aware that they are using RPC even if the syntax resembles a local procedure call.

A common pitfall is overly fine-grained RPC. For example, a file system that performs one RPC per byte would be unusable. A better design sends larger chunks, caches results, or changes the interface so that each request carries enough work to amortize network overhead.

4.4 RPC failure semantics

RPC introduces **partial failure**. In a local call, the caller and callee usually fail together because they are in the same process. In an RPC, the client can be alive while the server is dead; the server can execute the request and crash before sending a reply; the reply can be lost; or the server can be merely slow. A timeout tells the client that it has not received a reply; it does not tell the client whether the server executed the request.

This uncertainty creates three common semantic goals:

At least once	The system retries until the operation executes at least once. This is easy to approximate, but the operation may execute more than once.
At most once	The system ensures the operation executes no more than once, typically using request identifiers and duplicate suppression. It may still fail without execution.
Exactly once	The operation appears to execute exactly once. This is the desired user-level semantics for many applications, but it is difficult to implement in the presence of crashes, timeouts, and lost replies.

Deep Dive

The danger of automatic retry depends on the operation. Retrying `ReadAt(file, offset, length)` is often safe because it returns the same bytes if the file version has not changed. Retrying `TransferMoney(Alice, Bob, 100)` can be disastrous if the first request executed but the reply was lost. Distributed systems therefore prefer **idempotent** operations, or they attach unique request identifiers so the server can detect duplicates.

Algorithm 1: Client-Side RPC with Timeout and Duplicate-Safe Request ID

Input: Operation name op, arguments $args$, timeout Δ, retry limit k
Output: Reply from server or failure

```
requestId ← fresh unique identifier for i ← 1 to k do
    send (requestId, op, args) to server;
    wait up to Δ for a reply;
    if matching reply arrives then
        return reply;
return failure;
```

The algorithm is only safe if the server treats `requestId` correctly. For non-idempotent operations, the server must remember completed request identifiers and return the previous reply rather than executing the operation again. This memory is itself state, and it must be made durable or reconstructable if the guarantee must survive server crashes.

5 Distributed-System Challenges

5.1 Worse availability can result from more machines

It is tempting to assume that a distributed system is more available because there are more machines. This is only true if the system is designed so that surviving machines can take over the work of failed machines. A naive distributed system can be less available because it depends on every machine being alive. If any required component fails, the user cannot complete work.

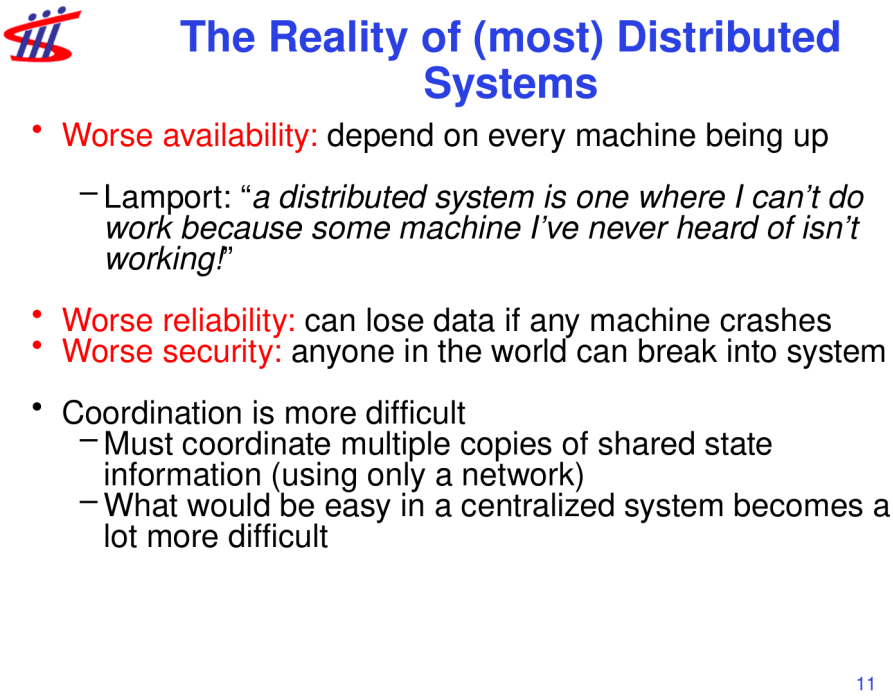


Figure 5: The reality of many distributed systems. The slide stresses worse availability, worse reliability, worse security, and harder coordination if the system depends on many machines without robust failure handling.

5.2 Reliability and failure-handling code

Distributed systems contain more failure-handling code than local systems. That code is often less tested than normal-case code because real failures are rare, expensive, and hard to reproduce. Consequently, a system may appear reliable until a real failure occurs, at which point the recovery path may overload another component, corrupt state, or trigger cascading failure.

Large organizations therefore perform failure drills. They deliberately disconnect a data center, shut down a storage subsystem, kill processes, or inject packet loss to test whether the system continues to operate. This engineering practice is sometimes called chaos engineering: controlled failure is introduced to discover hidden assumptions before an uncontrolled failure occurs.

5.3 Security

A distributed system typically has a larger attack surface than a single-machine program. It has network endpoints, authentication protocols, access-control rules, certificate management, service-to-service credentials, exposed dashboards, administrative APIs, and many software dependencies. Security is harder than reliability because an attacker actively searches for weaknesses rather than waiting for random accidents.

5.4 The core consistency question

The lecture's central image is a dog-sled team: many dogs provide more power than one, but the real challenge is making them run in the same direction. In technical terms, distributed systems must keep multiple copies of state consistent enough for the application.



Figure 6: The core challenge of distributed systems: coordinating independent components so that they act consistently rather than pulling in different directions.

Consistency is not one property. It is a spectrum of semantics. Some applications need strong linearizability; some need serializable transactions; some need close-to-open file consistency; some accept eventual consistency; and some expose conflicts to users. The correct design depends on the workload and user expectations.

6 Distributed File Systems

6.1 The goal: transparent remote files

A distributed file system provides transparent access to files stored on remote disks. Ideally, a program calls `open`, `read`, `write`, and `close` as if the file were local, while the file system turns those operations into network communication with a server.

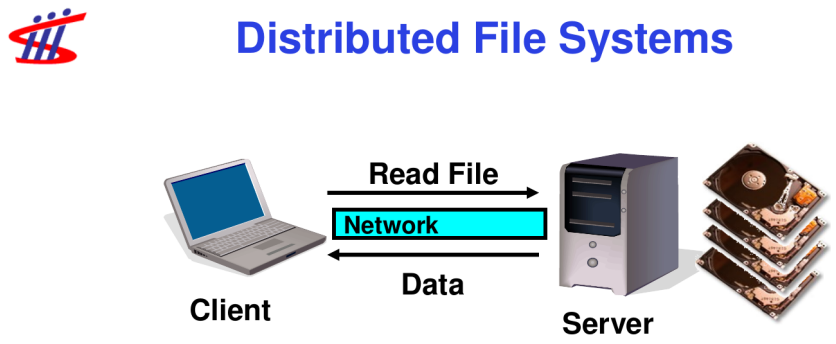


Figure 7: A distributed file system provides transparent access to a file stored on a remote disk. The client issues ordinary file operations; data and metadata are exchanged with a server across the network.

This is a powerful abstraction because it lets multiple clients share files and lets administrators centralize storage management. It is also a difficult abstraction because files are mutable shared objects, and clients often cache them for performance.

6.2 A simple design: forward every operation

The simplest distributed file system forwards every read and write to the server. The server stores the authoritative copy and can provide a completely consistent view to all clients, because all operations pass through one place.

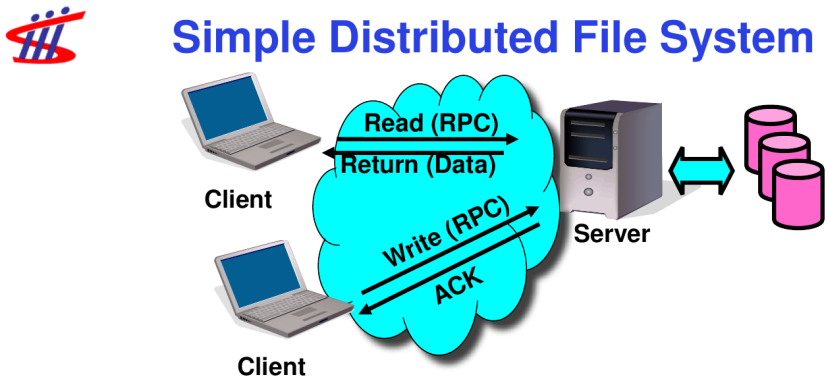


Figure 8: A simple distributed file system forwards every read and write to the server. This gives the server complete control over consistency, but it makes every operation pay network latency and can make the server a bottleneck.

The performance problem is severe. Local memory access may take nanoseconds, local disk access may take microseconds to milliseconds, and network access may add substantial latency and queueing. If every small operation crosses the network, applications slow down and the server becomes a bottleneck.

6.3 Caching to reduce network load

The standard performance technique is caching. If a client reads a file once, it stores the data locally. Future reads can be served from the client cache without network traffic. If a client opens, reads, writes, and closes a file entirely locally, performance can approach local-file-system performance.

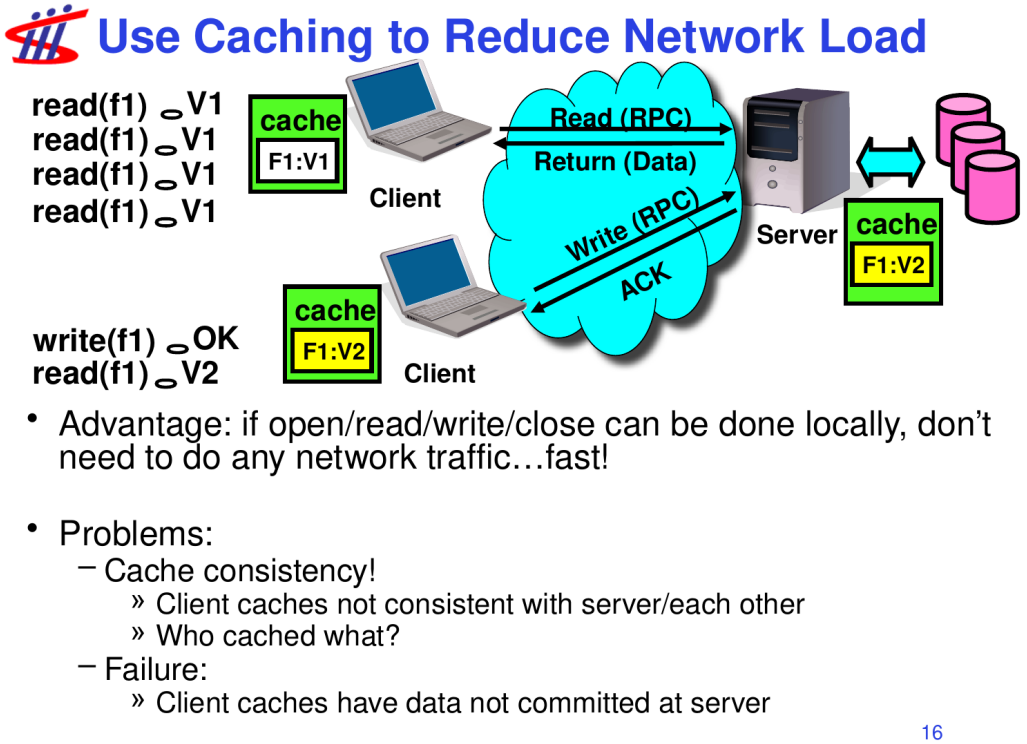


Figure 9: Client caching reduces network load but creates cache-consistency problems. The server, client caches, and other clients may temporarily disagree about which version of a file is current.

Caching creates two questions:

(1) **Write propagation:** when does a client send its local modifications to the server?

(2) **Cache invalidation:** when do other clients learn that their cached copies are stale?

Write-through caching sends modifications to the server immediately. Write-back caching delays propagation until a later time, such as file close, cache pressure, or periodic flush. Write-through improves freshness but sacrifices performance; write-back improves performance but increases inconsistency and risks losing dirty client data if the client crashes.

6.4 Block-level versus file-level network storage

Networked storage can appear at different abstraction levels. A storage-area network (SAN) exposes block devices over a network; clients may run their own file systems over remote blocks. This requires low latency because block operations are frequent and small. Network-attached storage (NAS) exposes files; the server runs the file system and clients send file operations such as open, read, write, and lookup. NFS and AFS are file-level distributed file systems and therefore fit the NAS model.

This distinction matters because the unit of sharing changes. A block-level system must coordinate low-level disk blocks and usually assumes a specialized storage network. A file-level system coordinates names, file contents, metadata, client caches, and user-level file semantics.

7 NFS: Simplicity, Statelessness, and Weak Consistency

7.1 Write-through caching

The Network File System (NFS) is a classic distributed file system that emphasizes simplicity and portability. Early NFS uses write-through caching: modified data is committed to the server's disk before the write is reported as complete to the client. This avoids some risks of losing dirty data on the client, but it loses many of the performance benefits of caching because writes still cross the network.

7.2 Weak cache consistency

NFS clients periodically poll the server to check whether cached data is still fresh. A typical rule is that if the client has not checked recently, in a tunable range such as 3 to 30 seconds, it asks the server whether the file changed. If one client modifies a file, the server learns of the update through write-through, but another client may continue using an old cached version until its validation interval expires.

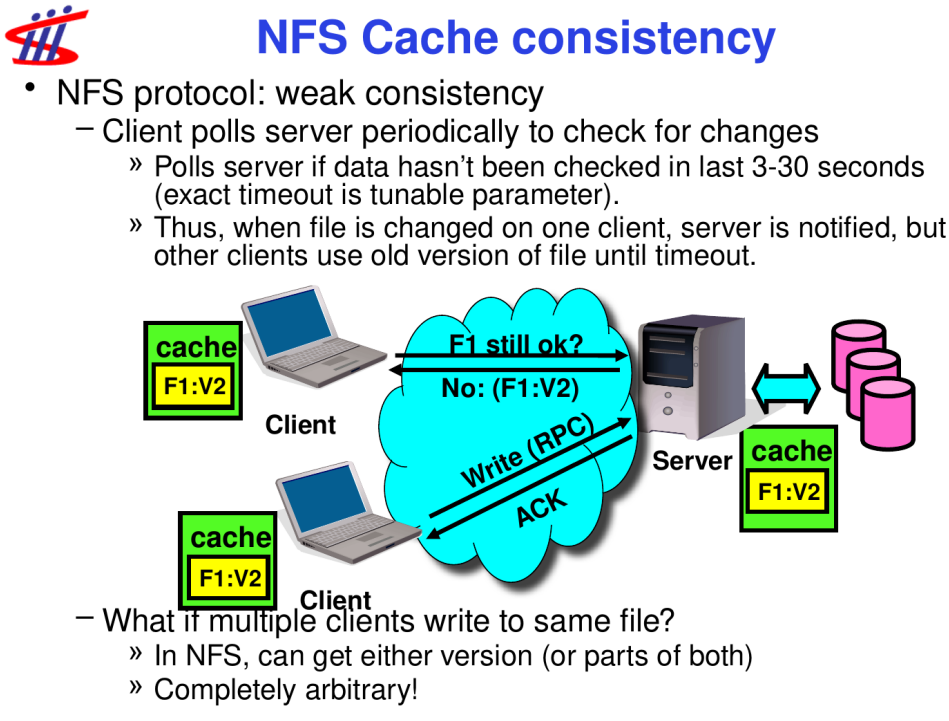


Figure 10: NFS weak consistency. A client checks with the server periodically rather than receiving immediate invalidations. Thus a file updated by one client may remain stale in another client's cache until the timeout expires.

This design is simple because the server does not have to remember every client that cached every file. It is also semantically weak. For some workloads, a 30-second window of staleness is acceptable; for collaborative editing, databases, build systems, or coordination files, it may be unacceptable.

Deep Dive

Polling is a trade-off between freshness and load. Poll too frequently and caching loses its purpose because the server receives constant validation traffic. Poll too infrequently and clients read stale data for too long. The NFS design chooses an engineering compromise rather than a strong consistency guarantee.

7.3 Multiple writers

If multiple clients write the same file concurrently, NFS can produce surprising results. Depending on timing, a reader may observe one version, another version, or in some cases pieces of multiple versions. NFS does not provide the same concurrency control one expects from a transactional database. Applications that require strong coordination must use additional locking or application-level protocols.

7.4 Pros and cons

NFS survived because it is simple, portable, and useful. It works well for many read-mostly workloads, software repositories, home directories, and environments where weak freshness is acceptable. Its weaknesses are exactly the costs of that simplicity: stale reads, polling overhead, server bottlenecks, sensitivity to network latency, and limited semantics under concurrent modification.

Strengths	Simple protocol, highly portable, easy to deploy, effective for many common workloads.
Weaknesses	Weak consistency, polling traffic, server bottleneck, poor behavior for concurrent writers, sensitivity to network latency.

8 AFS: Whole-File Caching, Callbacks, and Session Semantics

8.1 Design goals

The Andrew File System (AFS) was designed for much larger scale than a small NFS installation. It aimed at campus-scale or even global-scale file sharing, with many clients and servers. The key goal was to minimize server work per client operation. If thousands of machines cache the same file, even $O(1)$ server work per local read can become too expensive.



Andrew File System

- Developed at CMU in the late 1980s
- Global distributed file system
  - "One AFS", like "one Internet"
    - » Why would you want more than one?
- **LARGE** numbers of clients, servers
  - 1000 machines could cache a single file,
  - Most local, some (very) remote
- Goal: Minimize / eliminate work per client operation
  - O(1) work per client may just be too expensive!

20

Figure 11: AFS overview. AFS was designed as a global distributed file system with large numbers of clients and servers. Its goal is to minimize work per client operation.

8.2 AFS assumptions

AFS makes workload assumptions that strongly shape its design:

(1) Client machines have local disks and can cache files for long periods.

(2) Whole-file caching is often reasonable because many files are read in large sequential chunks.

(3) Write-write and write-read sharing are rare; most files are updated by one user on one machine.

(4) Reading from a local disk is cheaper than repeatedly fetching data from the network, especially under the hardware assumptions of the original design.

Read-read sharing is not a problem. If many clients read the same file, they can all cache it locally. The hard cases involve a writer and one or more readers, or multiple writers.

8.3 Whole-file caching

Instead of fetching small blocks on demand, AFS tends to fetch an entire file when it is opened. Later reads and writes operate on the local cached copy. This design is excellent when the client will read most of the file anyway and when local disk access is cheap. It reduces server load dramatically because repeated operations do not involve the server.

Whole-file caching is less attractive for huge files, random access to tiny ranges, or workloads with many concurrent writers. Later systems often use chunk-based designs as a compromise: a file is divided into large chunks, each of which can be cached and invalidated independently.

8.4 Callbacks

AFS improves cache consistency using callbacks. When a client caches a file, it registers with the server. The server promises to notify that client if another client modifies the file. The callback is effectively a cache-validity promise: as long as the callback has not been broken, the client can trust its cached copy.

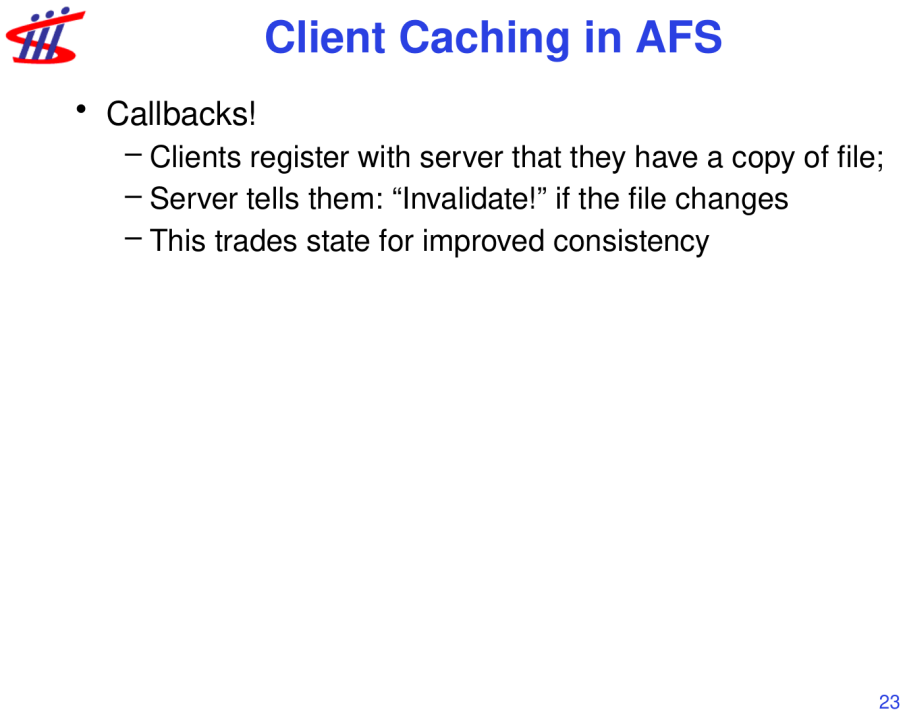


Figure 12: AFS callbacks. Clients register cached copies with the server. When a file changes, the server sends invalidation messages to clients that hold callbacks for that file.

Callbacks trade server state for reduced polling. NFS avoids maintaining per-client cache state and therefore uses polling. AFS maintains callback state and therefore can make cache validation event-driven. This is a classic distributed-systems trade-off: more state at the server can provide better performance and fresher caches, but that state must be recovered or reconstructed after failure.

9 File Access Consistency

9.1 One-copy semantics in a local Unix file system

In a local Unix file system, processes on the same machine share the kernel buffer cache. If one process writes a file, another process reading the same file usually sees the update quickly, often immediately according to the kernel's synchronization rules. Conceptually, the system tries to behave like there is one copy of the file.



File Access Consistency

- In UNIX local file system, concurrent file reads and writes have “sequential” consistency semantics
  - Each file read/write from user-level app is an atomic operation
    - » The kernel locks the file vnode
  - Each file write is immediately visible to all file readers
- Ideal: One Copy Semantics
- Neither NFS nor AFS provides such concurrency control
  - NFS: “sometime within 30 seconds”
  - AFS: session semantics for consistency

25

Figure 13: File access consistency. A local Unix file system can approximate one-copy semantics using one kernel and one buffer cache. Distributed file systems such as NFS and AFS weaken the semantics to improve performance and scalability.

Distributed file systems cannot cheaply preserve the same semantics. Multiple clients have separate memories, separate caches, separate clocks, and separate failures. The file system must choose weaker semantics or pay for stronger coordination.

9.2 NFS semantics

NFS offers weak timeout-based consistency. A write may reach the server, but another client’s cache may remain stale until its polling interval expires. The result is often summarized as “sometime within about 30 seconds,” though the exact value is tunable and workload-dependent.

9.3 AFS session semantics

AFS uses **session semantics**. A file write is visible to processes on the same client machine immediately because they share the local client cache. The write is not visible to processes on other machines until the writer closes the file and the client sends the modified file back to the server. After close, new opens can see the new version, but old opens may continue with the old version.



Session Semantics in AFS

- What it means:
  - A file write is visible to processes on the same box immediately, but not visible to processes on other machines until the file is closed
  - When a file is closed, changes are visible to new opens, but are not visible to "old" opens
  - Last writer (i.e. the last close()) wins
- Implementation
  - Dirty data are buffered at the client machine until file close, then flushed back to server, which leads the server to send "break callback" to other clients

27

Figure 14: AFS session semantics. Writes are buffered at the client and propagated to the server on close. The last client to close a concurrently modified file wins.

The phrase **last close wins** is important. If two clients open the same file, edit separate cached copies, and close at different times, the version closed last overwrites the server copy. This is not linearizable and not serializable in the database sense. It is acceptable only because AFS assumes concurrent writes are rare and because many users primarily edit their own files.



AFS: Cache Consistency Timeline

P_1	P_2	Cache	P_3	Cache	Server Disk	*Comments*
		Client$_1$		Client$_2$		
open(F)		-		-	-	File created
write(A)		A		-	-	
close()		A		-	A	
	open(F)	A		-	A	
	read() → A	A		-	A	
	close()	A		-	A	
open(F)		A		-	A	
write(B)		B		-	A	
	open(F)	B		-	A	Local processes
	read() → B	B		-	A	see writes immediately
	close()	B		-	A	
		B	open(F)	A	A	Remote processes
		B	read() → A	A	A	do not see writes...
		B	close()	A	A	
close()		B		~~A~~	B	... until close()
		B	open(F)	B	B	has taken place
		B	read() → B	B	B	
		B	close()	B	B	
		B	open(F)	B	B	
open(F)		B		B	B	
write(D)		D		B	B	
		D	write(C)	C	B	
		D	close()	C	C	
close()		D		~~C~~	D	
		D	open(F)	D	D	Unfortunately for P_3
		D	read() → D	D	D	the last writer wins
		D	close()	D	D	

Figure 50.3: **Cache Consistency Timeline**

Source:
Remzi, OS Book, "AFS"

Figure 15: AFS cache-consistency timeline. The timing diagram illustrates how cached versions, callback invalidations, opens, writes, and closes interact under session semantics.

9.4 Why not use ordinary file sharing for collaborative editing?

AFS-style semantics are not suitable for Google Docs-like collaborative editing, where many users may edit the same document concurrently and expect semantic merging. Collaborative editors typically implement application-level consistency: operation transformation, CRDTs, version histories, merge algorithms, or user-visible conflict resolution. The file system layer does not understand paragraphs, comments, spreadsheet cells, or source-code syntax; the application layer does.

10 Failure Handling in Distributed File Systems

10.1 Partial failure in a file system

Distributed file systems face many partial failures:

(1) A server crashes after performing disk I/O but before replying.

(2) A server crashes and loses volatile state.

(3) A client crashes with dirty data in its cache.

(4) A network link partitions clients from servers.

(5) A reply is lost, duplicated, or delayed.

(6) A client retries an operation that may already have executed.

A local file system can often assume the kernel has one coherent view of open files and buffer state. A distributed file system cannot.

10.2 NFS stateless server design

A central NFS design choice is server statelessness. Each request carries all information needed to execute the operation. A read request should specify the file identity, offset, and length, as in `ReadAt(inumber, position, length)`, rather than relying on a server-maintained open-file cursor such as `Read(openfile)`.



NFS Server Design Choices

- NFS servers are stateless; each request provides all arguments require for execution
  - E.g. reads include information for entire operation, such as `ReadAt(inumber,position)`, not `Read(openfile)`
  - No need to perform network open() or close() on file – each operation stands on its own
  - Server keeps no state about client, except as hints to help improve performance (e.g. a cache)
- Idempotent: Performing requests multiple times has same effect as performing it exactly once
  - Example: Server crashes between disk I/O and message send, client resend read, server does operation again
  - Example: Read and write file blocks: just re-read or re-write file block – no side effects
  - Example: What about "remove"? NFS does operation twice and second time returns an advisory error

33

Figure 16: NFS server design choices. NFS servers are stateless: each request provides all arguments needed for execution. Operations are designed to be idempotent when possible, making retry after crash or timeout safer.

Definition

An operation is **idempotent** if performing it multiple times has the same effect as performing it once. Idempotency is crucial for retry after timeouts because the client may not know whether the server executed the first request.

10.3 Why explicit offsets matter

Consider two designs for reading a file:

Stateful cursor read	`Read(openFile, length)` depends on a server-maintained current position. If a reply is lost and the client retries, the second execution may read the next block rather than the intended block.
Positioned read	`ReadAt(fileId, offset, length)` makes the cursor explicit. Retrying the same request reads the same region, assuming the file version has not changed.

Algorithm 2: Idempotent Positioned Read in an NFS-Style Server

Input: File identifier fid, byte offset off, byte count n
Output: Bytes from range $[off, off + n)$ or an error

```
file ← look up stable file handle fid;
if file does not exist then
    return error;
data ← read bytes [off, off + n) from stable storage or cache;
return data;
```

This is retry-friendly because the server does not advance hidden state. The client can resend the same request after a timeout. Similar logic applies to writes: `WriteAt(fileId, offset, data)` is more retry-friendly than `Append(data)`, because append depends on the current file length and can append twice if retried incorrectly.

10.4 NFS mount behavior: wait or return error

If an NFS server crashes, the client has two broad choices. A hard-mounted file system may hang until the server returns, preserving the illusion that the file operation will eventually complete. A soft-mounted file system may return an error to the application after a timeout. Neither choice is universally correct. Waiting forever may freeze applications. Returning errors may break programs that were not written to handle remote-file failures.

This illustrates false transparency. A program that believes it is reading a local file may be surprised by network errors, server crashes, or indefinite blocking. Distributed systems cannot always hide the network safely.

10.5 AFS failure handling and soft state

AFS servers maintain callback state, but callback state is **soft state**. If an AFS server crashes, it may lose the table recording which clients cache which files. Losing this table is not a permanent correctness failure; it can be reconstructed by asking clients to revalidate or report their cached files.

Definition

Hard state is state whose loss can violate correctness unless it is recovered from durable storage. **Soft state** can be reconstructed after loss; losing it may hurt performance or temporarily weaken guarantees, but it does not permanently destroy the system's meaning.

The callback table in AFS is soft state because the server can rebuild it. The actual file contents and committed metadata are hard state because losing them loses user data. Good distributed-system design tries to minimize hard state and make as much state as possible reconstructable.

10.6 Client crashes

Client crashes are dangerous when clients use write-back caching. If a client modifies a file locally and crashes before closing or flushing, the server may never see the update. AFS accepts this risk because it optimizes for common workloads where the client is the natural owner of the session and write conflicts are rare. Systems with stronger durability requirements must force data to stable storage sooner or replicate dirty state elsewhere.

11 Disconnected Operation: Coda

Coda extends AFS with disconnected operation. The idea is that a laptop should continue working when the network disappears. Coda treats disconnection as a normal mode rather than an exceptional failure.

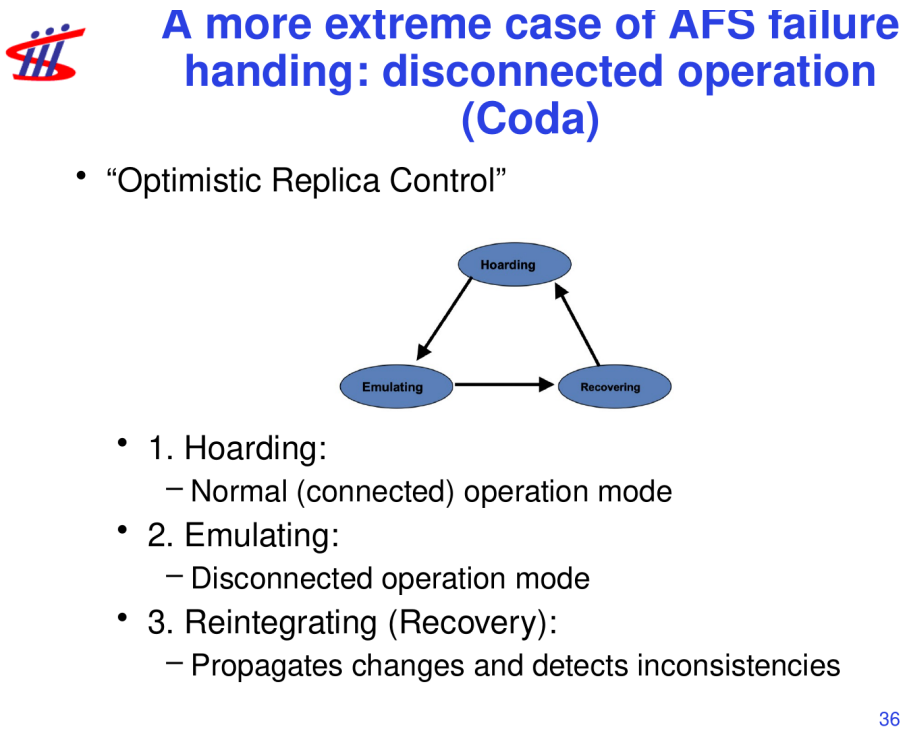


Figure 17: Coda disconnected operation. Coda uses optimistic replica control with three modes: hoarding while connected, emulating server behavior while disconnected, and reintegrating changes when connectivity returns.

11.1 Hoarding

In the hoarding phase, the client is connected and tries to cache files that may be useful later. Because cache space is limited and disconnection may be unpredictable, the system uses a mixture of user preferences and observed access patterns. Hoarding is a prediction problem: the system must guess which files the user will need while offline.

11.2 Emulation

In the emulation phase, the client is disconnected. If an application accesses a file that is not in the local cache, the operation fails because the server cannot be contacted. If the file is cached, the client services reads and writes locally. Modifications are recorded in a persistent client modification log. Obsolete log entries, such as updates to a file that is later deleted, can be compressed or removed.

11.3 Reintegration

When the client reconnects, Coda reintegrates local changes with the servers. The client ships the replay log; servers replay operations for each volume and detect conflicts. Coda focuses especially on write-write conflicts. If conflict resolution succeeds, the logs can be discarded. If

not, the system saves the conflicting state and asks for help, often from the user or a higher-level application.

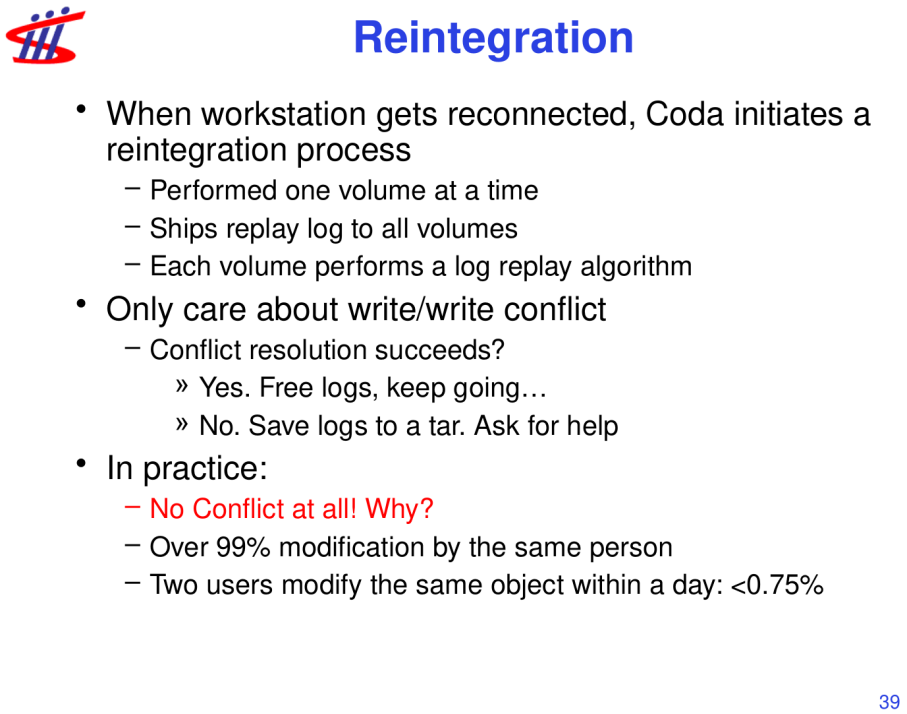


Figure 18: Coda reintegration. After reconnection, Coda ships replay logs to servers, replays operations volume by volume, detects write-write conflicts, and either frees the logs or exposes unresolved conflicts.

Coda is an example of optimistic design. It assumes conflicts are rare, permits work to proceed, and resolves conflicts later. This is similar in spirit to optimistic concurrency control in databases and to Git-style distributed development. The performance and availability benefits are large when conflicts are rare; the cost is that unresolved conflicts must be exposed rather than hidden.

12 Consistency Models and Design Philosophy

12.1 ACID versus BASE as design attitudes

Database transactions are often summarized by ACID: atomicity, consistency, isolation, and durability. These properties are extremely valuable, but enforcing them across many machines can require expensive coordination. Many large distributed systems therefore adopt a weaker design attitude often summarized as BASE: Basically Available, Soft state, and Eventual consistency.

BASE is not a precise theorem and not a drop-in replacement for ACID. It is a reminder that many distributed systems prefer availability and scalability over immediate global consistency. AFS is a good example: clients may temporarily observe stale or conflicting file versions, but the system continues operating and eventually converges to some stored version, unless a conflict must be resolved by a user or application.

12.2 Where should conflicts be resolved?

A low-level file system understands bytes, file names, metadata, and perhaps file chunks. It does not understand the semantics of a word-processing document, a spreadsheet, a source-code

refactoring, or a shopping cart. Therefore, when conflicts require semantic understanding, it is often better to expose them to the application layer.

This principle explains why Git does not pretend that all edits are serializable file-system transactions. It stores versions, detects conflicts, and lets tools or users merge. It also explains why collaborative editors implement specialized merge algorithms instead of relying on NFS or AFS semantics.

12.3 The design checklist

When designing or analyzing a distributed system, ask the following questions:

(1) What is the authoritative state?

(2) Which copies are caches, replicas, or hints?

(3) Which state is hard and must survive crashes?

(4) Which state is soft and can be reconstructed?

(5) Which operations are idempotent?

(6) Which operations need request identifiers or duplicate suppression?

(7) When do writes become visible to other clients?

(8) What does a timeout mean, and who decides whether to retry?

(9) What happens if the client crashes?

(10) What happens if the server crashes?

13 Worked Examples

13.1 Example 1: stale reads in NFS

Clients C_1 and C_2 both cache file F at version V_1. Client C_1 writes version V_2 and the server records it. Client C_2 has not validated its cache recently, so it continues reading V_1. After its validation timeout expires, C_2 asks the server whether its cached attributes are current. The server reports that the file changed, and C_2 fetches V_2.

The surprising part is not eventual convergence, but the fact that the old version can be observed after the new version exists on the server. This is the price paid for client caching without immediate invalidation.

13.2 Example 2: last close wins in AFS

Clients C_1 and C_2 both open file F when the server version is A. Client C_1 edits its local copy into version B. Client C_2 edits its local copy into version C. If C_1 closes first, the server becomes B and callbacks may invalidate other cached copies. If C_2 later closes, it writes back C, and the server becomes C.

The final state discards B unless the user or application preserved it elsewhere. This behavior is acceptable only under the assumption that concurrent writes are rare.

13.3 Example 3: retry-safe positioned reads

A client sends `ReadAt(file=17, offset=4096, length=4096)`. The server reads the bytes and sends the reply, but the reply is lost. The client times out and resends the same request. If the file version has not changed, the server returns the same bytes. Repeating the request does not move a hidden cursor.

Now compare `Read(file=17, length=4096)` where the server stores the current cursor. If the first request succeeded but the reply was lost, the retry reads the next block. If the first request never arrived, the retry reads the intended block. From the client's perspective, these cases look the same. This is why NFS-style protocols encode operation state explicitly.

13.4 Example 4: duplicate suppression for a non-idempotent RPC

Suppose a payment service receives `Charge(account, amount)`. Retrying this RPC blindly can charge twice. A safer interface includes a unique client-generated operation identifier:

```
# Server-side sketch.
completed = {}  # maps request_id -> previous reply

def charge(request_id, account, amount):
    if request_id in completed:
        return completed[request_id]

    # The following update and the record of request_id must be atomic
    # if duplicate suppression must survive server crashes.
    debit(account, amount)
    reply = {"ok": True}
    completed[request_id] = reply
    return reply
```

Listing 2: Duplicate suppression for a non-idempotent operation

The comment is the key point. If the server crashes after debiting the account but before recording `request_id`, a retry may debit again. Strong duplicate suppression for non-idempotent operations requires durable state, transactions, or a carefully designed log.

14 Summary

Distributed systems build one coherent service out of independent computers. They are useful because they provide more computational power, information sharing, modularity, incremental scaling, and administrative independence. They are difficult because there is no shared memory, no single global view, and many forms of partial failure.

RPC is the basic communication abstraction that turns message exchange into procedure-like calls. It improves programmability through stubs, marshalling, unmarshalling, and IDLs, but it does not make remote calls equivalent to local calls. A remote call can time out even though it executed, can execute without returning, or can be retried unsafely.

Distributed file systems show the trade-offs concretely. A simple server-forwarding design is consistent but slow. Caching improves performance but creates invalidation and failure problems. NFS chooses simplicity, write-through caching, client polling, weak consistency, stateless servers, and idempotent requests. AFS chooses whole-file caching, callbacks, session semantics, write-back, and soft state to improve scalability. Coda extends AFS by supporting disconnected operation through hoarding, emulation, and reintegration.



Summary: Distributed System Challenges

- Multiple processes
- Multiple copies of data
  - Cache to reduce traffic
  - Replication to improve availability
- Consistency issues
  - Am I reading / writing stale data?
- Especially: Partial failures

- Distributed System Theory: Full of Impossibility Results!
- In practice: how to make it “mostly” possible

40

Figure 19: Summary of distributed-system challenges: multiple processes, multiple copies of data, caching, replication, stale reads and writes, partial failures, and the practical need to make theoretically difficult problems mostly manageable.

The enduring lesson is that distributed systems are not made correct by hiding the network. They are made correct by designing explicit semantics for communication, caching, consistency, retry, and recovery.

15 Quizzes & Exercises

15.1 Short-Answer Quiz

Answer each question in two to three sentences.

1. What is a distributed system, and why is the phrase "single-system image" important?
2. Why can adding more machines make availability worse rather than better?
3. What problem does RPC solve compared with raw socket programming?
4. What are marshalling and unmarshalling in an RPC system?
5. Why is an RPC timeout ambiguous?
6. What is an idempotent operation, and why does it matter for retries?
7. Why does a simple distributed file system that forwards every read and write to the server provide strong consistency but poor performance?
8. What cache-consistency problem is created when distributed file-system clients cache file data?
9. What is the NFS 3-30 second validation idea, and what trade-off does it represent?
10. Why does NFS try to keep servers stateless?
11. What is the main difference between NFS polling and AFS callbacks?
12. What are AFS session semantics?
13. What does "last close wins" mean, and when is it dangerous?
14. What is soft state? Give the AFS callback table as an example.
15. What are the three phases of Coda disconnected operation?

15.2 Applied Exercises

1. Design an RPC interface for a file-system `read` operation. Make it retry-safe and explain why it is safer than a server-maintained cursor.
2. Design an RPC interface for an `append` operation. Explain why append is not naturally idempotent and propose a request-ID-based solution.
3. Suppose two NFS clients cache file `config.txt`. Client C_1 writes a new version at time $t = 0$, while C_2 validates only every 30 seconds. Draw a timeline showing what C_2 may read between $t = 0$ and $t = 30$.
4. Suppose two AFS clients open the same file, edit it, and close it in opposite orders. Explain which version remains on the server and why.
5. A distributed build system stores source files in a distributed file system. Which semantics would be safer: NFS weak consistency, AFS session semantics, or an application-level version-control protocol? Justify your answer.
6. For a collaborative document editor, explain why last-close-wins is insufficient and what kind of application-level merge information is needed.

7. Classify each piece of state as hard or soft: file contents, inode metadata, NFS server file cursor, AFS callback table, client-side dirty cache, DNS cache entry, duplicate request table for a payment RPC.

8. Compare a hard-mounted NFS directory that waits forever with a soft-mounted NFS directory that returns errors. Which is safer for a database file? Which is more convenient for a read-only software repository?

9. Explain how Coda can allow useful work while disconnected and why reintegration can still fail.

10. Construct a failure scenario in which automatic retry is safe, and another in which automatic retry causes an incorrect result.

15.3 Essay Questions

1. **Scalability versus consistency.** Compare NFS and AFS. How do write-through, polling, whole-file caching, callbacks, and session semantics affect scalability and correctness?

2. **False transparency.** RPC and distributed file systems both try to hide communication. Discuss which aspects can be hidden safely and which must be exposed to programmers.

3. **Failure handling as design.** Explain why statelessness, idempotency, soft state, and request identifiers are central distributed-system techniques.

4. **Modularity.** Use e-commerce services and car electronic control units as examples to explain why distributed systems are often chosen for organizational and modularity reasons, not only for performance.

5. **Disconnected operation.** Analyze Coda's hoarding, emulation, and reintegration phases. How is this similar to optimistic concurrency control or Git-style development?

15.4 Suggested Answers to the Short-Answer Quiz

1. A distributed system is a collection of independent computers that appears to users as one coherent system. A single-system image matters because the system's value often comes from hiding distribution behind a simpler abstraction.

2. More machines mean more components that can fail. Availability improves only if the system is designed so that remaining components can continue or take over after a failure.

3. RPC raises the interface from untyped byte streams to procedure-like calls with typed arguments and return values. Generated stubs and an IDL reduce manual parsing, byte-order bugs, and protocol mismatches.

4. Marshalling converts typed arguments into a canonical byte representation; unmarshalling reconstructs typed values from those bytes. RPC systems use these steps to communicate across languages, machines, and address spaces.

5. A timeout means only that the client did not receive a reply in time. The request may not have arrived, may have executed with the reply lost, or may still be executing slowly.

6. An idempotent operation has the same effect if performed multiple times as if performed once. Idempotency matters because clients often retry after timeouts without knowing whether the first request executed.

7. Forwarding every operation to the server gives the server one authoritative view of the file system. It performs poorly because every operation pays network latency and the server can become a bottleneck.

8. If clients cache data, one client may modify a file while another still has the old version. The system must decide when writers send updates and when other caches are invalidated.

9. NFS clients poll the server periodically, often after a few to about thirty seconds, to validate cached data. Frequent polling improves freshness but increases server load; infrequent polling improves performance but allows stale reads.

10. Stateless servers can crash and restart without recovering complex per-client state. Because each request contains all needed arguments, clients can often retry operations after recovery.

11. NFS polling makes clients ask whether cached data is still valid. AFS callbacks make the server notify clients when cached data becomes invalid, trading server state for less polling and better scalability.

12. AFS session semantics make writes visible locally immediately but visible to other machines only when the file is closed and written back. New opens after the close see the new version; old opens may continue with an old version.

13. Last close wins means that if multiple clients modify the same file, the client that closes last overwrites the server copy. It is dangerous for collaborative editing because earlier updates can be silently lost.

14. Soft state is reconstructable state whose loss does not permanently violate correctness. The AFS callback table is soft because a restarted server can ask clients to revalidate or report cached files.

15. Coda uses hoarding while connected, emulation while disconnected, and reintegration after reconnection. Hoarding predicts needed files, emulation serves cached files locally, and reintegration replays logged updates and detects conflicts.